

Exp9: Implement a Kubernetes ingress controller to manage external access to services.

Step 0: Prerequisites Check

Run:

```
kubectl version --client
```

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl version --client
Client Version: v1.35.3
Kustomize Version: v5.7.1
cse@cse-B760M-H:~/CT_LAB/exp9$
```

```
minikube status
```

```
cse@cse-B760M-H:~/CT_LAB/exp9$ minikube status
minikube
type: Control Plane
host: Stopped
kubelet: Stopped
apiserver: Stopped
kubeconfig: Stopped

cse@cse-B760M-H:~/CT_LAB/exp9$
```

If Minikube not running:

```
minikube start
```

```
cse@cse-B760M-H:~/CT_LAB/exp9$ minikube start
🌻 minikube v1.38.1 on Ubuntu 24.04
🔧 Using the docker driver based on existing profile
👍 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.50 ...
🔄 Restarting existing docker container for "minikube" ...
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔍 Verifying Kubernetes components...
   ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: default-storageclass, storage-provisioner
🏁 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
cse@cse-B760M-H:~/CT_LAB/exp9$
```

🚀 Step 1: Enable Ingress Controller

```
minikube addons enable ingress
```

```
cse@cse-B760M-H:~/CT_LAB/exp9$ minikube addons enable ingress
💡 ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
■ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
■ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
■ Using image registry.k8s.io/ingress-nginx/controller:v1.14.3
🔍 Verifying ingress addon...
🌟 The 'ingress' addon is enabled
cse@cse-B760M-H:~/CT_LAB/exp9$
```

Check:

kubectl get pods -n ingress-nginx

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl get pods -n ingress-nginx
NAME                                READY   STATUS    RESTARTS   AGE
ingress-nginx-admission-create-brgfg 0/1     Completed 0           71s
ingress-nginx-admission-patch-q5qwd 0/1     Completed 1           71s
ingress-nginx-controller-596f8778bc-6xmkm 1/1     Running   0           71s
cse@cse-B760M-H:~/CT_LAB/exp9$
```

👉 Wait until all pods are Running

Step 2: Create Deployment

kubectl create deployment webapp --image=nginx

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl create deployment webapp --image=nginx
deployment.apps/webapp created
cse@cse-B760M-H:~/CT_LAB/exp9$
```

kubectl get deployments

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
myapp     3/3     3             3           24h
webapp    1/1     1             1           2m20s
cse@cse-B760M-H:~/CT_LAB/exp9$
```

🔗 Step 3: Expose as Service

kubectl get services

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl get services
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes ClusterIP   10.96.0.1    <none>        443/TCP    15d
cse@cse-B760M-H:~/CT_LAB/exp9$
```

kubectl expose deployment webapp --port=80 --type=NodePort

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl expose deployment webapp --port=80 --type=NodePort
service/webapp exposed
cse@cse-B760M-H:~/CT_LAB/exp9$
```

Check:

kubectl get services

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl get services
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes    ClusterIP     10.96.0.1     <none>         443/TCP          15d
webapp        NodePort      10.100.76.179 <none>         80:32469/TCP     4s
cse@cse-B760M-H:~/CT_LAB/exp9$
```

Step 4: Create Ingress File

nano ingress.yaml

Paste:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webapp-ingress
spec:
  rules:
  - host: webapp.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: webapp
            port:
              number: 80
```

Save: CTRL + O → Enter → CTRL + X

```
GNU nano 7.2
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webapp-ingress
spec:
  rules:
  - host: webapp.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: webapp
            port:
              number: 80
```

Step 5: Apply Ingress

kubectl get ingress

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl get ingress
No resources found in default namespace.
cse@cse-B760M-H:~/CT_LAB/exp9$
```

kubectl apply -f ingress.yaml

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/webapp-ingress created
cse@cse-B760M-H:~/CT_LAB/exp9$
```

Check:

kubectl get ingress

```
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl get ingress
NAME          CLASS   HOSTS          ADDRESS      PORTS    AGE
webapp-ingress  nginx   webapp.local    80           3s
cse@cse-B760M-H:~/CT_LAB/exp9$
```

Step 6: Configure Local DNS (VERY IMPORTANT)

Get IP of minikube:

minikube ip

```
cse@cse-B760M-H:~/CT_LAB/exp9$ minikube ip
192.168.49.2
cse@cse-B760M-H:~/CT_LAB/exp9$
```

Copy the IP address

Open hosts file:

sudo nano /etc/hosts

```
GNU nano 7.2
127.0.0.1 localhost
127.0.1.1 cse-B760M-H

# The following lines are desirable for IPv6 capable hosts
::1      ip6-localhost ip6-loopback
fe00::0  ip6-localnet
ff00::0  ip6-mcastprefix
ff02::1  ip6-allnodes
ff02::2  ip6-allrouters
```

Add:

<minikube-ip> webapp.local

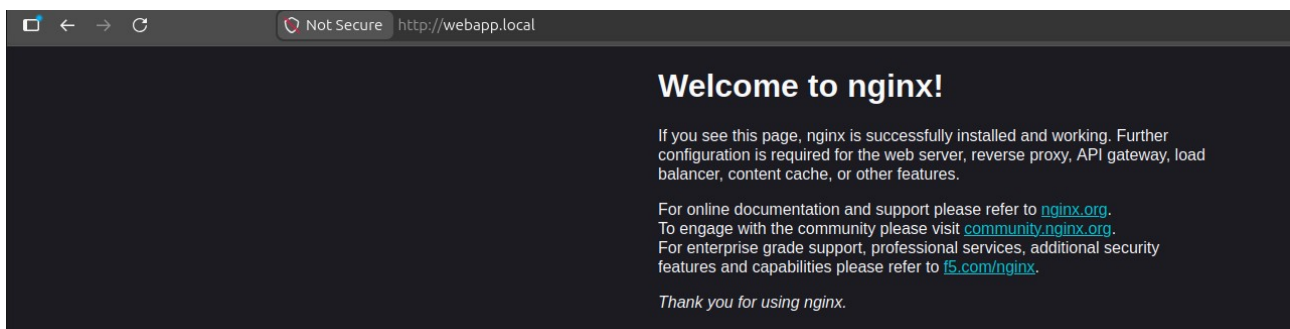
```
GNU nano 7.2
127.0.0.1 localhost
127.0.1.1 cse-B760M-H
192.168.49.2 webapp.local
# The following lines are desirable for IPv6 capable hosts
::1      ip6-localhost ip6-loopback
fe00::0  ip6-localnet
ff00::0  ip6-mcastprefix
ff02::1  ip6-allnodes
ff02::2  ip6-allrouters
```

Save and exit

Step 7: Access Application

Option 1 (Browser)

<http://webapp.local>



Option 2 (Terminal)

`curl http://webapp.local`

👉 You should see NGINX welcome page

```
cse@cse-B760M-H:~/CT_LAB/exp9$ curl http://webapp.local
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
cse@cse-B760M-H:~/CT_LAB/exp9$
```

Step 8: Verify Everything

```
kubectl get pods
kubectl get svc
kubectl get ingress
```

```

cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myapp-6849dcb48b-ncw2v             1/1     Running   1 (23h ago) 24h
myapp-6849dcb48b-thkg9             1/1     Running   1 (23h ago) 24h
myapp-6849dcb48b-vbvt7             1/1     Running   1 (23h ago) 24h
webapp-645b5b46cd-8cpkq            1/1     Running   0           18m
cse@cse-B760M-H:~/CT_LAB/exp9$
cse@cse-B760M-H:~/CT_LAB/exp9$
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl get services
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
kubernetes      ClusterIP   10.96.0.1     <none>        443/TCP          15d
webapp          NodePort    10.100.76.179 <none>        80:32469/TCP     14m
cse@cse-B760M-H:~/CT_LAB/exp9$
cse@cse-B760M-H:~/CT_LAB/exp9$
cse@cse-B760M-H:~/CT_LAB/exp9$ kubectl get ingress
NAME            CLASS    HOSTS           ADDRESS        PORTS    AGE
webapp-ingress  nginx    webapp.local    192.168.49.2   80       10m
cse@cse-B760M-H:~/CT_LAB/exp9$

```

🎯 Expected Output

- ✓ NGINX page loads via custom domain
- ✓ No need to use NodePort directly
- ✓ Ingress routing works